# UNIT-III

- **Linked Lists using C&Python:**
- **Singly linked lists**
- Representation in memory,
- Algorithms of several operations:
- Traversing, Searching, Insertion into, Deletion from linked list;
- Linked representation of Stack and Queue.
- **Doubly Linked List:** operations on it and algorithmic analysis.
- **Circular Linked List:** all operations on it. Applications of Linked List.

# Disadvantages of Arrays

## 1.Static size

Array size must be decided in advance. Cannot grow or shrink during program execution.

## 2. Memory wastage

If declared size is larger than needed, unused memory is wasted.

## 3.Insertion and deletion are difficult

Elements must be shifted to insert or delete, which takes **more time (O(n))**.

## 4.Homogeneous elements only

An array can store **only one data type** (all integers, all floats, etc.).

## 5. contiguous memory

Elements are stored in continuous (adjacent) memory locations.

# Advantages of Linked List

## 1. Dynamic size

Linked list size **need not be fixed in advance**.

It can **grow or shrink during program execution**.

## 2. No memory wastage

Memory is allocated **only when a new node is created**.

Hence, **unused memory is not wasted**.

## 3. Easy insertion and deletion

Elements can be inserted or deleted **without shifting other elements**.

## 4. Can store different data using structures

Linked lists can store **different data** using structures.

More **flexible than arrays**.

## 5. No need for contiguous memory

Nodes are stored in **different memory locations**.

# What is Linked List?

- A linked list is a linear data structure where nodes are connected using pointers and allocated in dynamic memory.
- In a **linked list**, elements are **not stored in continuous memory** like arrays.
- Each element (called a **node**) is stored **anywhere in memory** and connected using **pointers (links)**.
- Every node contains:
  - **Data**
  - **Address (pointer) of the next node**

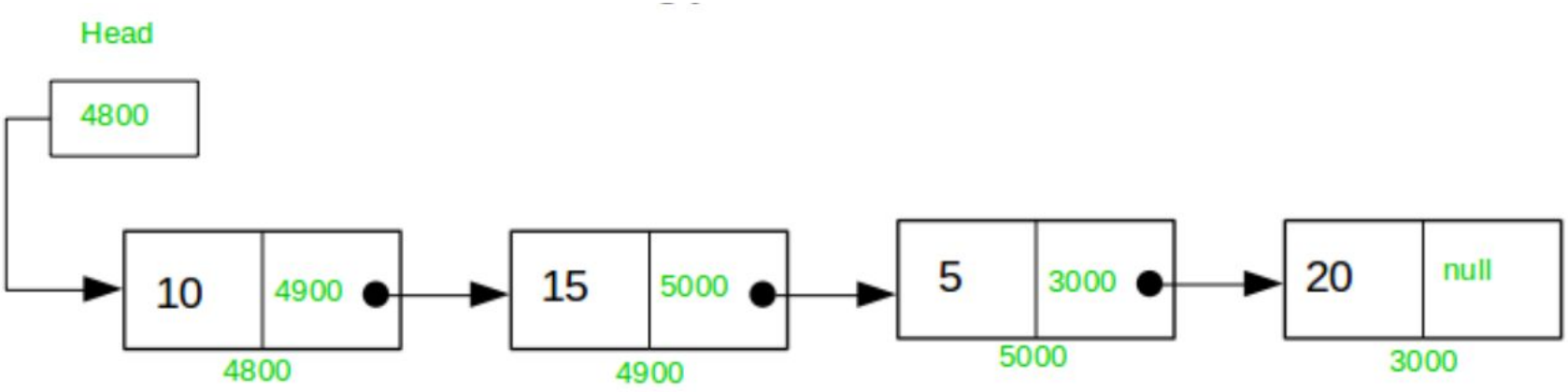


# Representation in memory

A **singly linked list** is stored in **non-contiguous memory locations**.

Each element (called a **node**) contains:

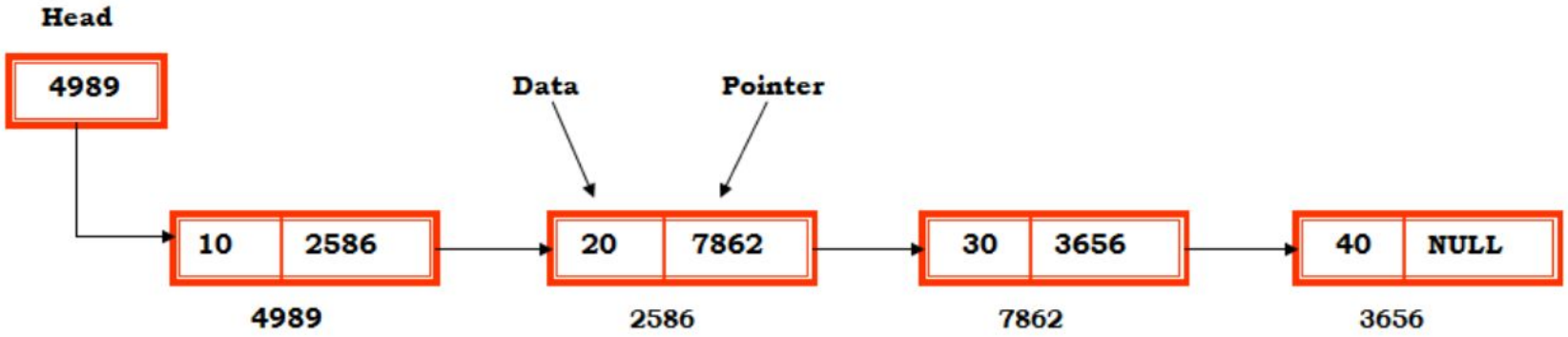
- **Data field** → stores the value
- **Next pointer** → stores the **address of the next node**



# 1.Singly Linked List (Navigation is Forward only)

**Each node points to the next node.**

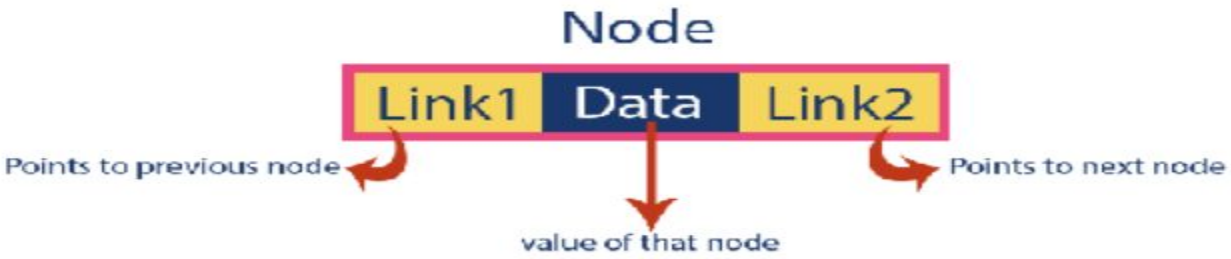


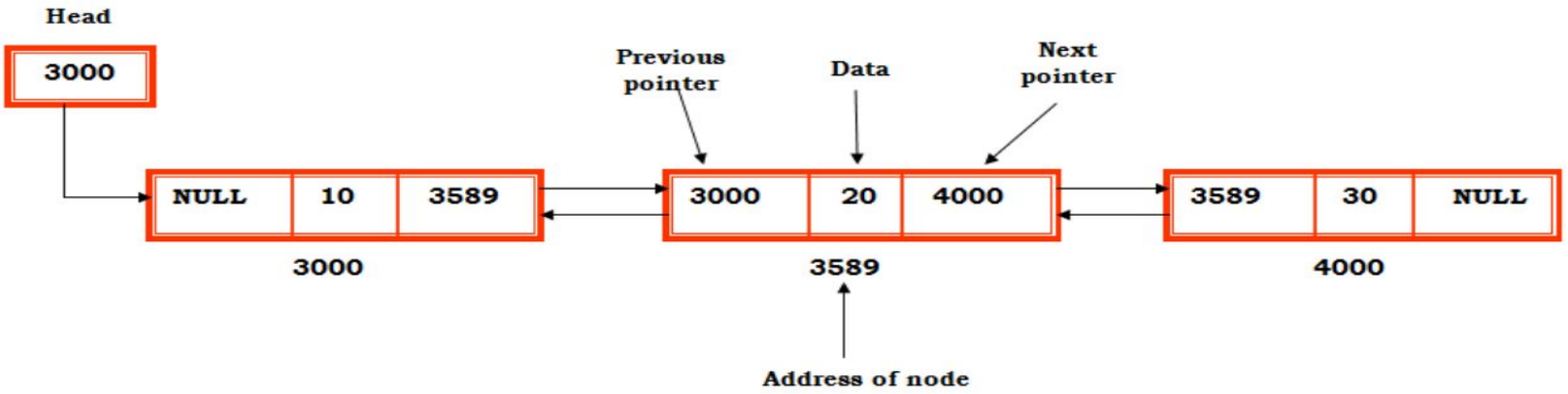



# 2.Double Linked List (Forward and Backward Navigation is possible)

**Each node has two pointers:**

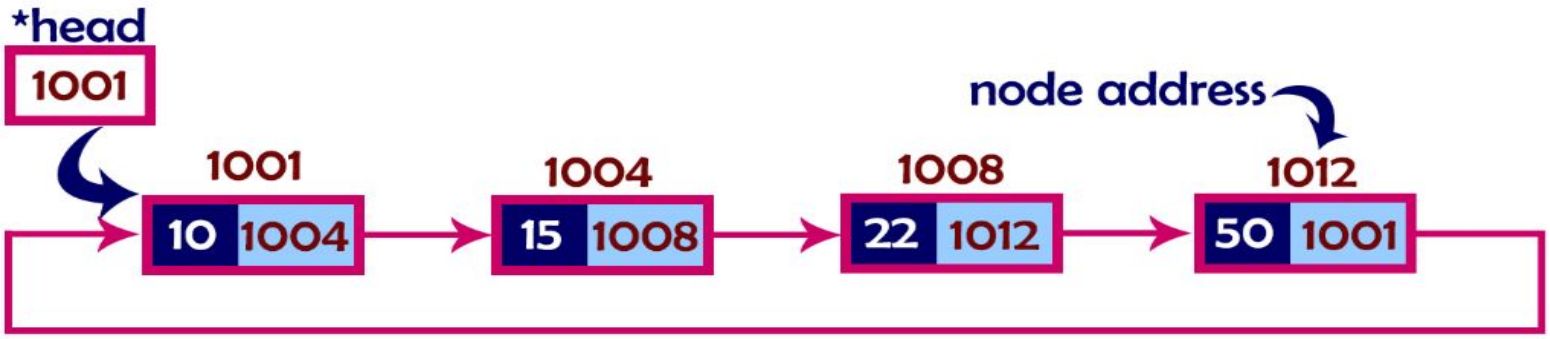
**one to the next node and one to the previous node.**



# 3.Circular Linked List (Last element is linked to the first element)

A circular linked list is a linked list in which the last node points back to the first node instead of NULL, the traversal in this linked list will go on forever until it is broken



# Linked List Operations

## 1. Insertion (Adding a Node)

You can add a node in three places:

- **At the beginning** → The new node becomes the first node.
- **At the end** → The new node is added after the last node.
- **At a specific position** → The new node is placed between two existing nodes.

## 2. Deletion (Removing a Node)

You can remove a node in three ways:

- **From the beginning** → The first node is removed, and the next node becomes the new head.
- **From the end** → The last node is removed.
- **By value** → The node with a specific value is found and removed.

## 3. Searching (Finding a Node)

checking whether a particular Node is present in the linked list.

## 4. Traversal (Displaying Elements)

Start from the first node and visit each node one by one until the end. Print each node's value.

- Start from the first node (head)
- Move node by node using the next pointer
- Continue until the end (NULL)
- Print each node’s value

## 5. Reversal (Reversing the List)

Reversal means changing the direction of links in a linked list so that:

- Last node becomes first
- First node becomes last
- All next pointers are reversed

# Creating a Node in Singly Linked List (SLL) in C

```
// Structure for a node
 struct Node
{
      int data;
      struct Node* next;
};
```

A **node** in a **singly linked list (SLL)** consists of:

1. **Data** – Stores the value.
2. **Pointer (Next)** – Points to the next node in the list.

# Memory allocation to a new node

- Memory for a node in a singly linked list is allocated dynamically using **malloc().**

- **sizeof(struct node)** gives the required memory size.

- The returned pointer is stored in a newnode pointer.

```
struct node *newnode;

newnode = (struct node*) malloc(sizeof(struct
 node));

 newnode->data = 10;   // assign value
 newnode->next = NULL; // next pointer set to
 NULL
```

# Algorithm: Create a Node and insert a node at end in SLL

**Step 1:** Start

**Step 2**: Allocate memory for a new node using malloc().

**Step 3:** Read the element value from the user.

**Step 4:** Store the value in
**newnode → data = ele**

**Step 5:** Set **newnode → next = NULL**

**Step 6:** Check if the list is empty (**head == NULL**)

If yes, set
**head = newnode**
**temp = newnode**

**Step 7:**

Otherwise (list not empty)
Link the new node at the end using
temp → next = newnode
temp = newnode

**Step 8:**

Print **"New node is created successfully."**

**Step 9:** Stop

# Algorithm: Delete First Node in a SLL

**Step 1:** Start

**Step 2:** Check if the list is empty (head == NULL)

If **yes**, print **"List is Empty"** and **stop**.

**Step 3:** Set temp = head.

**Step 4:** Display the data of the node to be deleted

Print **"Deleted Node is = temp → data"**.

**Step 5:** Move head to the next node

Set head = head → next.

**Step 6:** Free the memory of the deleted node

Call free(temp).

**Step 7:** Stop.

# Algorithm: Display Elements of a SLL

**Step 1:** Start

**Step 2:** Set temp = head.

**Step 3:** Check if the list is empty (head == NULL)

If **yes**, print **"List is Empty"** and **stop**.

**Step 4:** Print the heading **"List Elements:"**.

**Step 5:** Traverse the list using a loop while (temp != NULL)

Print temp → data followed by " -> ".

Move to the next node using temp = temp → next.

**Step 6:** After the loop ends, print **"NULL"**.

**Step 7:** Stop.

**//Implement SLL using Dynamic memory**

**Creating and Inserting a Node in a Singly Linked List (at the end)**

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
   int data;
   struct node* next;
}

*head = NULL, *temp, *newnode;
```

```
void create_Node()
{
    int ele;
   newnode = (struct node*)malloc(sizeof(struct
     node));
    printf("Enter element: ");
   scanf("%d", &ele);
   newnode->data = ele;
   newnode->next = NULL;
   if (head == NULL)
   {
      head = newnode;
      temp = newnode;
   }
   else
   {
      temp->next = newnode;
      temp = newnode;
   }
   printf("New node is created successfully.\n");
}
```

```
void delete_Node()
{
   if (head == NULL)
   {
      printf("List is Empty \n");
      return;
   }

    temp = head;
    printf("Deleted Node is = %d\n",
    temp->data);

   head = head->next;
   free(temp);   // ? memory freed
}
```

```
void display()
{
   temp = head;

   if (head == NULL)
   {
      printf("List is Empty \n");
      return;
   }
   printf("List Elements: ");
   while (temp != NULL)
   {
      printf("%d -> ", temp->data);
      temp = temp->next;
   }
   printf("NULL\n");
}
```

```
void main()
{
   int choice;

   while (1)
   {
      printf("\n1.Create\n2.Display\n3.Delete\n
      4.Exit\n");
       printf("Enter your choice: ");
       scanf("%d", &choice);

       switch (choice)
       {
           case 1: create_ode();
            break;
           case 2: display();
            break;
           case 3: delete_Node();
            break;
           case 4:
          printf("Exit from Linked List\n");
              exit(0);
           default:
              printf("Please enter a valid
       choice.\n");
        }
    }
}
```

C:\Users\ANIL\Desktop\FINAL PROGRAMS\Singlelinkedfinal.exe

```
                                SINGLE LINKED LIST MENU

1.Create
2.Display
3.Delete
4.Exit
Enter your choice: 1
Enter element: 10
New node is created successfully.

                                SINGLE LINKED LIST MENU

1.Create
2.Display
3.Delete
4.Exit
Enter your choice: 1
Enter element: 20
New node is created successfully.

                                SINGLE LINKED LIST MENU

1.Create
2.Display
3.Delete
4.Exit
Enter your choice: 1
Enter element: 30
New node is created successfully.
```

```
                                SINGLE LINKED LIST MENU

1.Create
2.Display
3.Delete
4.Exit
Enter your choice: 2
List Elements: 10 -> 20 -> 30 -> NULL

                                SINGLE LINKED LIST MENU

1.Create
2.Display
3.Delete
4.Exit
Enter your choice: 3
Deleted Node is = 10

                              SINGLE LINKED LIST MENU

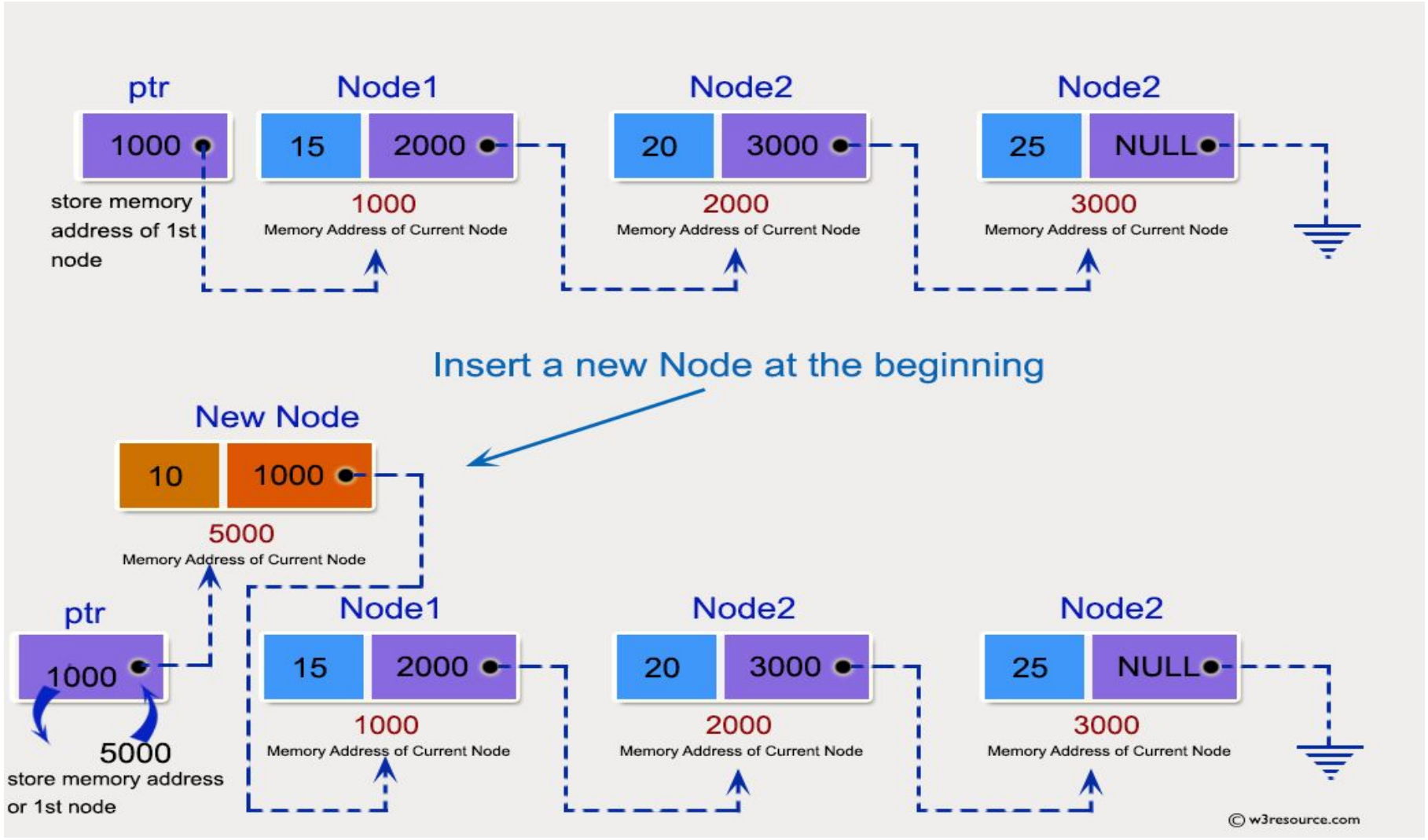
1.Create
2.Display
3.Delete
4.Exit
Enter your choice: 3
Deleted Node is = 20

                              SINGLE LINKED LIST MENU

1.Create
2.Display
3.Delete
4.Exit
Enter your choice: 3
Deleted Node is = 30
```

```
                                SINGLE LINKED LIST MENU
1.Create
2.Display
3.Delete
4.Exit
Enter your choice: 3
List is empty, nothing to delete.

                                SINGLE LINKED LIST MENU
1.Create
2.Display
3.Delete
4.Exit
Enter your choice: 4
Exit from Linked List
```


ptr
Node1
Node2
Node2
1000
15
2000
20
3000
25
NULL
store memory address of 1st node
1000
Memory Address of Current Node
2000
Memory Address of Current Node
3000
Memory Address of Current Node
Insert a new Node at the beginning
New Node
10
1000
5000
Memory Address of Current Node
ptr
Node1
Node2
Node2
1000
15
2000
20
3000
25
NULL
5000
store memory address or 1st node
1000
Memory Address of Current Node
2000
Memory Address of Current Node
3000
Memory Address of Current Node
© w3resource.com

```
void  insert_Beg()
{
    int ele;
   newnode = (struct node*)malloc(sizeof(struct node));
    printf("Enter element: ");
   scanf("%d", &ele);

    newnode->data = ele;     // store data
   newnode->next = head;    // link to old first node
   head = newnode;          // update head

   printf("Node inserted at Begin successfully\n");
}
```